In [2]:
```
!pip install gensim
```

```
Requirement already satisfied: gensim in C:\Users\manis\AppData\Local\Python\pyth
oncore-3.12-64\Lib\site-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in C:\Users\manis\AppData\Local\Pyth
on\pythoncore-3.12-64\Lib\site-packages (from gensim) (2.5.4)
Requirement already satisfied: scipy>=1.7.0 in C:\Users\manis\AppData\Local\Pytho
n\pythoncore-3.12-64\Lib\site-packages (from gensim) (1.18.1)
Requirement already satisfied: smart_open>=1.8.1 in C:\Users\manis\AppData\Local
\Python\pythoncore-3.12-64\Lib\site-packages (from gensim) (9.0.1)
Requirement already satisfied: wrapt in C:\Users\manis\AppData\Local\Python\pytho
ncore-3.12-64\Lib\site-packages (from smart_open>=1.8.1->gensim) (2.5.2)
```

In [1]:
```
# 1. Import necessary packages
import numpy as np
import pandas as pd
import re

from gensim.models import Word2Vec
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Dense

# 2. Load the IMDB dataset
data = pd.read_csv('IMDB Dataset.csv')

print(data.head())

# 3. Preprocess the text
stop_words = {'the', 'is', 'a', 'an', 'and', 'to', 'of', 'in', 'it', 'this', 'th

def preprocess(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', '', text)
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return words

sentences = data['review'].apply(preprocess).tolist()

# Convert sentiment labels into numbers
labels = data['sentiment'].map({
    'positive': 1,
    'negative': 0
}).values

# 4. Split training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    sentences, labels,
    test_size=0.2,
    random_state=42
)

# 5. Train CBOW Word2Vec model
cbow_model = Word2Vec(
    sentences=X_train,
    vector_size=100,
    window=5,
```

```
    min_count=2,
    sg=0,
    epochs=10
)

# 6. Convert documents into embeddings
def document_vector(words):
    vectors = [
        cbow_model.wv[word]
        for word in words
        if word in cbow_model.wv
    ]

    if len(vectors) == 0:
        return np.zeros(100)

    return np.mean(vectors, axis=0)

X_train_vec = np.array([document_vector(x) for x in X_train])
X_test_vec = np.array([document_vector(x) for x in X_test])

# 7. Build Feedforward Neural Network
model = Sequential([
    Input(shape=(100,)),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# 8. Train the model
history = model.fit(
    X_train_vec, y_train,
    epochs=5,
    batch_size=32,
    validation_split=0.1
)

# 9. Evaluate the model
loss, accuracy = model.evaluate(X_test_vec, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")
```

```
                                                review sentiment
0  One of the other reviewers has mentioned that ...  positive
1  A wonderful little production. <br /><br />The...  positive
2  I thought this was a wonderful way to spend ti...  positive
3  Basically there's a family where a little boy ...  negative
4  Petter Mattei's "Love in the Time of Money" is...  positive
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/5
1125/1125 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8336 - loss: 0.3770 - va
l_accuracy: 0.8580 - val_loss: 0.3334
Epoch 2/5
1125/1125 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8516 - loss: 0.3401 - va
l_accuracy: 0.8637 - val_loss: 0.3258
Epoch 3/5
1125/1125 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8559 - loss: 0.3336 - va
l_accuracy: 0.8627 - val_loss: 0.3188
Epoch 4/5
1125/1125 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8581 - loss: 0.3294 - va
l_accuracy: 0.8583 - val_loss: 0.3238
Epoch 5/5
1125/1125 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8596 - loss: 0.3261 - va
l_accuracy: 0.8635 - val_loss: 0.3188
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8523 - loss: 0.3329
Test Loss: 0.33290642499923706
Test Accuracy: 85.22999882698059 %
```

In [ ]: